COMSATS UNIVERSITY ISLAMABAD



CSC211-Data Structures

Lab 5:

Stack

Name	Muhammad Awais
Registration	FA25-BSE-057
Class	BSE-3B
Instructor's Name	Mr. Zahid Anwar
Date	October 10, 2026

Activity 1:

```
#include <iostream>
using namespace std;
class Stack
{
private:
   int top;
   char values[];

 public:
   Stack(int x)
   {
      top = -1;
      values[x];
   }

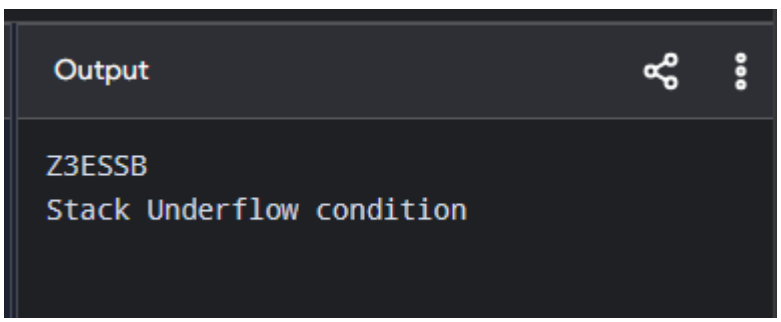
   void push(char x)
   {
       if(top==10)
          cout<<"\nStack overflow condition";
          return;
       else
          values[++top] = x;
   }

   char pop()
   {
       if(top == -1){
          cout<<"\nStack Underflow condition";
```

```
            return ;
        }
        else
            return values[top--];
    }
};
int main() {
    Stack* myStack = new Stack(10);
    myStack->push('B');
    myStack->push('S');

    myStack->push('S');
    myStack->push('E');
    myStack->push('3');
    myStack->push('Z');

    cout<<myStack->pop();
    cout<<myStack->pop();
    cout<<myStack->pop();
    cout<<myStack->pop();
    cout<<myStack->pop();
    cout<<myStack->pop();
    cout<<myStack->pop();
}
```



Activity 2

```
#include <iostream>
using namespace std;

struct Node
{
       char data;
       Node* next;
};

class LinkedStack
{
      private:
            Node* head;
      public:
             LinkedStack()
             {
                head = NULL;
             }

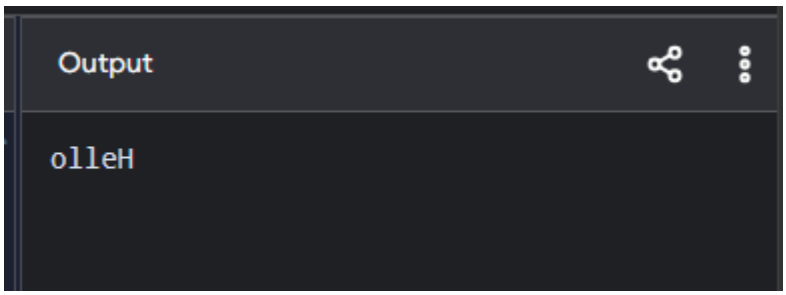
      void push(char c)
      {
          Node* temp = new Node;
          temp->data = c;
          temp->next = head;
          head = temp;
      }

      char pop()
      {
           if(head == NULL)
           {
             cout << "\nStack is Empty ....Underflow
Condition\n";
             return '\0';
           }
           else
           {
                Node* temp = head;
                char T = temp->data;
                head = temp->next;
                delete temp;
                return T;
           }
      }
};

int main()
{
```

```
        LinkedStack myStack;
        myStack.push('H');
        myStack.push('e');
        myStack.push('l');
        myStack.push('l');
        myStack.push('o');

        cout << myStack.pop();
        cout << myStack.pop();
        cout << myStack.pop();
        cout << myStack.pop();
        cout << myStack.pop();
}
```



Activity 3

```
#include <iostream>
#include <cstring>
using namespace std;
class Stack
{
private:
    int top;
    char values[18];

 public:
    Stack()
    {
       top = -1;

    }

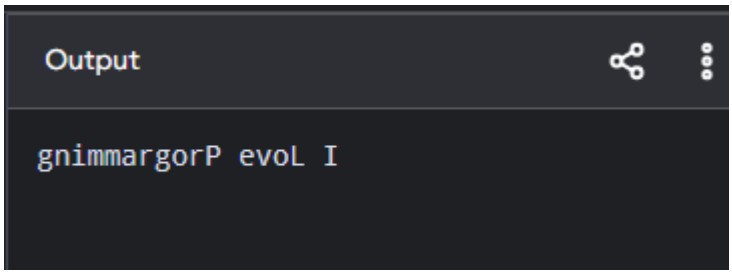
    void push(char x)
    {
        if(top==17) {
            cout<<"\nStack overflow condition";
            return;
    }
        else
            values[++top] = x;
```

```
    }

    char pop()
    {
        if(top == -1){
            cout<<"\nStack Underflow condition";
            return 0 ;
       }
       else
            return values[top--];
  }
};
int main() {

    Stack s1;

    char str[] = "I Love Programming";
    int len = strlen(str);
    for(int i=0; i<len; i++)
        s1.push(str[i]);
    for(int i=0; i<len; i++)
       cout << s1.pop();
    return 0;
}
```



Activity 4

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isBalanced(string expr) {
    stack<char> s;
    char ch;

    for (int i = 0; i < expr.length(); i++) {

        if (expr[i] == '(' || expr[i] == '[' || expr[i] ==
'{') {
```

```
            s.push(expr[i]);
            continue;
        }

        if (s.empty())
            return false;

        switch (expr[i]) {
            case ')':
                ch = s.top();
                s.pop();
                if (ch == '{' || ch == '[')
                    return false;
                break;

            case '}':
                ch = s.top();
                s.pop();
                if (ch == '(' || ch == '[')
                    return false;
                break;

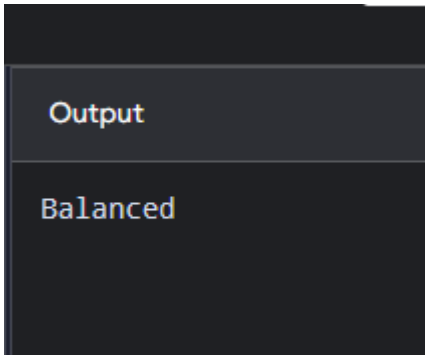
            case ']':
                ch = s.top();
                s.pop();
                if (ch == '(' || ch == '{')
                    return false;
                break;
        }
    }

    // If stack is empty at the end, all brackets matched
successfully
    return s.empty();
}

int main() {
    string expr = "[{}(){()}]";

    if (isBalanced(expr))
        cout << "Balanced";
    else
        cout << "Not Balanced";

    return 0;
}
```



Lab Task 1:

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int getPriority(char ch) {
    if (ch == '[') return 3;
    if (ch == '{') return 2;
    if (ch == '(') return 1;
    return 0;
}

bool isBalanced(string expr) {
    stack<char> s;
    char ch;

    for (int i = 0; i < expr.length(); i++) {

        if (expr[i] == '(' || expr[i] == '[' || expr[i] == '{') {
            // 2. ADD THIS PRIORITY CHECK BEFORE PUSHING
            if (!s.empty()) {
```

```
                if (getPriority(expr[i]) >
getPriority(s.top())) {
                    return false;
                }
            }
            s.push(expr[i]);
            continue;
        }

        if (s.empty())
            return false;

        switch (expr[i]) {
            case ')':
                ch = s.top();
                s.pop();
                if (ch == '{' || ch == '[')
                    return false;
                break;

            case '}':
                ch = s.top();
                s.pop();
                if (ch == '(' || ch == '[')
                    return false;
                break;

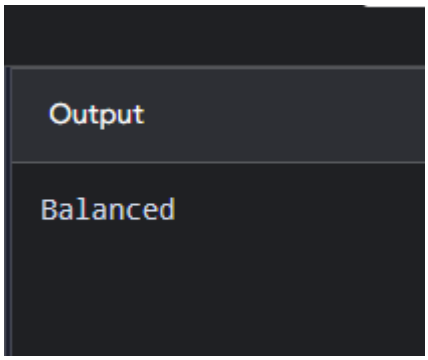
            case ']':
                ch = s.top();
                s.pop();
                if (ch == '(' || ch == '{')
```

```
                    return false;
                break;
            }
        }

    return s.empty();
}

int main() {
    string expr = "[{( )}]";
 if (isBalanced(expr))
        cout << "Balanced";
    else
        cout << "Not Balanced";

    return 0;
}
```



Lab Task 2:

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    char data;
    Node* link;
};
```

```
// Dynamic Stack Implementation
class LinkedStack {
private:
    Node* head;
public:
    LinkedStack() {
        head = NULL;
    }

    void push(char e) {
        Node* temp = new Node;
        temp->data = e;
        temp->link = head;
        head = temp;
    }

    char pop() {
        if (head == NULL) return '\0';
        Node* temp = head;
        char T = temp->data;
        head = temp->link;
        delete temp;
        return T;
    }

    char peek() {
        if (head == NULL) return '\0';
        return head->data;
    }

    bool isEmpty() {
        return head == NULL;
    }

    // Destructor to clean up memory
    ~LinkedStack() {
        while (!isEmpty()) {
            pop();
        }
    }
};

// Function to determine operator precedence
int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return -1;
}
```

```
// Function to convert Infix expression to Postfix
string infixToPostfix(string s) {
    LinkedStack st;
    string result = "";

    for (int i = 0; i < s.length(); i++) {
        char c = s[i];

        // If the character is an operand (letter or digit),
add it to output
        if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')
|| (c >= '0' && c <= '9')) {
            result += c;
        }
        // If the character is '(', push it to the stack
        else if (c == '(') {
            st.push('(');
        }
        // If the character is ')', pop and output from stack
until '(' is found
        else if (c == ')') {
            while (!st.isEmpty() && st.peek() != '(') {
                result += st.pop();
            }
            if (!st.isEmpty()) {
                st.pop(); // Remove '(' from stack
            }
        }
        // If an operator is encountered
        else {
            while (!st.isEmpty() && precedence(c) <=
precedence(st.peek())) {
                result += st.pop();
            }
            st.push(c);
        }
    }

    // Pop all remaining elements from the stack
    while (!st.isEmpty()) {
        result += st.pop();
    }

    return result;
}

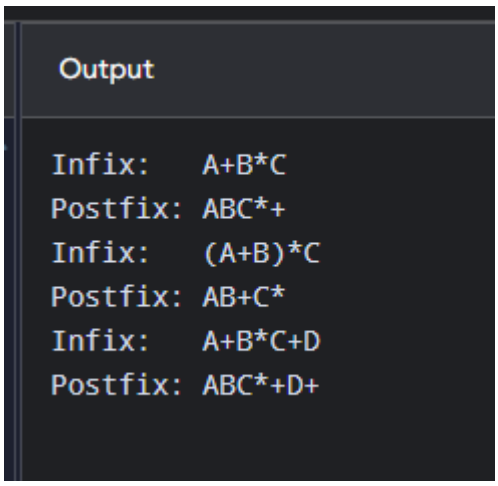
int main() {
    // Test cases
    string testCases[] = {
        "A+B*C",
        "(A+B)*C",
```

```
        "A+B*C+D",
    };

    int numTests = 3;
    for (int i = 0; i < numTests; i++) {
        cout << "Infix:   " << testCases[i] << endl;
        cout << "Postfix: " << infixToPostfix(testCases[i]) <<
endl;

    }

    return 0;
}
```



Lab task 3

```
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>
#include <cmath>
#include <stdexcept>
#include <cstdlib>

using namespace std;

// Dynamic Stack implemented using a Linked List
class LinkedStack {
private:
    struct Node {
        double data;
        Node* next;
        Node(double val) : data(val), next(nullptr) {}
    };
```

```
    Node* topNode;
    int count;

public:
    LinkedStack() : topNode(nullptr), count(0) {}

    // Destructor to free remaining memory
    ~LinkedStack() {
        while (!isEmpty()) {
            pop();
        }
    }

    void push(double val) {
        Node* newNode = new Node(val);
        newNode->next = topNode;
        topNode = newNode;
        count++;
    }

    double pop() {
        if (isEmpty()) {
            throw runtime_error("Stack underflow");
        }
        Node* temp = topNode;
        double val = temp->data;
        topNode = topNode->next;
        delete temp;
        count--;
        return val;
    }

    bool isEmpty() const {
        return topNode == nullptr;
    }

    int size() const {
        return count;
    }
};

class PostfixEvaluator {
private:
    unordered_map<string, double> variables;

    bool isOperator(const string& token) {
        return token == "+" || token == "-" || token == "*" ||
token == "/" || token == "^";
    }
```

```
    double applyOperator(double a, double b, const string& op)
{
        if (op == "+") return a + b;
        if (op == "-") return a - b;
        if (op == "*") return a * b;
        if (op == "/") {
            if (b == 0) throw runtime_error("Division by
zero");
            return a / b;
        }
        if (op == "^") return pow(a, b);
        throw runtime_error("Unknown operator: " + op);
    }

    bool isNumber(const string& s) {
        if (s.empty()) return false;
        char* p;
        strtod(s.c_str(), &p);
        return *p == 0;
    }

public:
    void setVariable(const string& name, double value) {
        variables[name] = value;
    }

    double evaluate(const vector<string>& tokens) {
        LinkedStack stack;

        for (const string& token : tokens) {
            if (isOperator(token)) {
                if (stack.size() < 2) {
                    throw runtime_error("Invalid postfix
expression: insufficient operands");
                }
                // Pop right operand first, then left operand
                double b = stack.pop();
                double a = stack.pop();

                double result = applyOperator(a, b, token);
                stack.push(result);
            }
            else {
                double val;
                if (isNumber(token)) {
                    val = stod(token);
                } else if (variables.find(token) !=
variables.end()) {
                    val = variables[token];
                } else {
```

```
                    throw runtime_error("Undefined variable or
invalid token: " + token);
                }
                stack.push(val);
            }
        }

        if (stack.size() != 1) {
            throw runtime_error("Invalid postfix expression:
malformed stack state");
        }

        return stack.pop();
    }
};

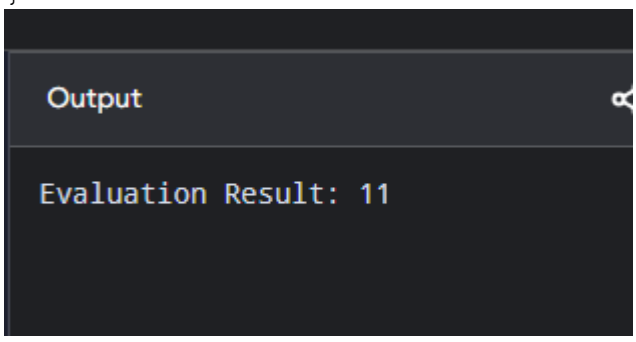
int main() {
    PostfixEvaluator evaluator;

    // Set variable values
    evaluator.setVariable("a", 4.0);
    evaluator.setVariable("b", 2.0);
    evaluator.setVariable("c", 3.0);

    // Expression: a b * c +  (Equivalent to: (4 * 2) + 3)
    vector<string> postfixTokens = {"a", "b", "*", "c", "+"};

    try {
        double result = evaluator.evaluate(postfixTokens);
        cout << "Evaluation Result: " << result << endl;
    } catch (const exception& e) {
        cerr << "Error: " << e.what() << endl;
    }

    return 0;
}
```



Additional tasks

Task 1

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

string simplifyPath(string path) {
  stack<string> s;
  string folder = "";

  for (int i = 0; i <= path.length(); i++) {

    if (i == path.length() || path[i] == '/') {

      if (folder == "" || folder == ".") {

      } else if (folder == "..") {

        if (!s.empty())
          s.pop();

      } else {
        s.push(folder);
      }

      folder = "";
    } else {
      folder += path[i];
```

```
    }
  }
  string result = "";

  stack<string> temp;

  while (!s.empty()) {
    temp.push(s.top());
    s.pop();
  }

  while (!temp.empty()) {
    result += "/" + temp.top();
    temp.pop();
  }
  if (result == "")
    result = "/";
  return result;
}
int main() {
  string path;
  cout << "Enter UNIX file path: ";
  getline(cin, path);
  cout << "Canonical path: " << simplifyPath(path);
  return 0;
}
```

```
c:\data-structures-cpp\labs\lab-05>cd "c:\data
additional-task-1
Enter UNIX file path: /a/./b/../../c//
Canonical path: /c
c:\data-structures-cpp\labs\lab-05>
```

Task 2

```
#include <cctype>
#include <iostream>
#include <stack>
#include <string>

using namespace std;

string decodeString(string s) {
  stack<int> countStack;
  stack<string> stringStack;

  string currentStr = "";
  int currentNum = 0;

  for (int i = 0; i < s.length(); i++) {
    char c = s[i];

    if (isdigit(c)) {

      currentNum = currentNum * 10 + (c - '0');
    }
```

```
  else if (c == '[') {
    countStack.push(currentNum);
    stringStack.push(currentStr);

    currentNum = 0;
    currentStr = "";
  }

  else if (c == ']') {
    int repeatTimes = countStack.top();
    countStack.pop();

    string prevStr = stringStack.top();
    stringStack.pop();

    string multipliedStr = "";
    for (int k = 0; k < repeatTimes; k++) {
      multipliedStr += currentStr;
    }

    currentStr = prevStr + multipliedStr;
  }

  else {
    currentStr += c;
  }
}

return currentStr;
```

```
}

int main() {
  string sample1 = "3[a2[c]]";

  cout << "Input 1: " << sample1 << "\nOutput 1: " << decodeString(sample1) << "\n\n";
  return 0;
}
```

```
c:\data-structures-cpp\labs\lab-05>cd "c
additional-task-1
Input 1: 3[a2[c]]
Output 1: accaccacc
```

Task 3

```
#include <algorithm>
#include <iostream>
#include <stack>
#include <vector>

using namespace std;

int largestRectangleArea(vector<int>& heights) {
  stack<int> s;
  int maxArea = 0;
  int n = heights.size();

  for (int i = 0; i <= n; i++) {

    int currentHeight = (i == n) ? 0 : heights[i];
```

```
    while (!s.empty() && currentHeight < heights[s.top()]) {

      int h = heights[s.top()];
      s.pop();

      int w;
      if (s.empty()) {
        w = i;
      } else {
        w = i - s.top() - 1;
      }
      maxArea = max(maxArea, h * w);
    }
    s.push(i);
  }

  return maxArea;
}
int main() {
  vector<int> heights = {2, 1, 5, 6, 2, 3};
  cout << "Histogram heights: [2, 1, 5, 6, 2, 3]\n";
  cout << "Largest Rectangle Area: " << largestRectangleArea(heights) << "\n";
  return 0;
}
```

```
c:\data-structures-cpp\labs\lab-05>cd "c:\data-stru
ta-structures-cpp\labs\lab-05\"additional-task
Histogram heights: [2, 1, 5, 6, 2, 3]
Largest Rectangle Area: 10
```

Task 4

```
#include <iostream>
#include <stack>
#include <vector>

using namespace std;

bool canReorderTrains(int n, vector<int>& desired) {
  stack<int> siding;

  int incomingTrain = 1;

  for (int i = 0; i < n; i++) {
    int targetTrain = desired[i];

    while (siding.empty() || siding.top() != targetTrain) {

      if (incomingTrain > n) {
        return false;
      }

      siding.push(incomingTrain);
      incomingTrain++;
    }

    siding.pop();
  }

  return true;
```

```
}

int main() {
  int n = 5;

  vector<int> desired1 = {2, 1, 5, 4, 3};
  cout << "Desired: [2, 1, 5, 4, 3] -> ";
  if (canReorderTrains(n, desired1)) {
    cout << "YES\n";
  } else {
    cout << "NO\n";
  }

  vector<int> desired2 = {3, 1, 2, 5, 4};
  cout << "Desired: [3, 1, 2, 5, 4] -> ";
  if (canReorderTrains(n, desired2)) {
    cout << "YES\n";
  } else {
    cout << "NO\n";
  }

  return 0;
}
```

```
c:\data-structures-cpp\labs\lab-05>cc
tional-task
Desired: [2, 1, 5, 4, 3] -> YES
Desired: [3, 1, 2, 5, 4] -> NO
```

Task 5

```
#include <iostream>
#include <stack>
#include <vector>

using namespace std;

vector<int> nextGreaterElements(vector<int>& nums) {
  int n = nums.size();

  vector<int> result(n, -1);

  stack<int> waitingRoom;

  for (int i = 0; i < 2 * n; i++) {

    int currentIndex = i % n;
    int currentNumber = nums[currentIndex];

    while (!waitingRoom.empty() && currentNumber > nums[waitingRoom.top()]) {

      int waitingIndex = waitingRoom.top();
      result[waitingIndex] = currentNumber;
      waitingRoom.pop();
    }

    if (i < n) {
      waitingRoom.push(currentIndex);
    }
```

```
  }

  return result;
}

int main() {
  vector<int> nums = {1, 2, 1};
  vector<int> result = nextGreaterElements(nums);

  cout << "Input:  [";
  for (int i = 0; i < nums.size(); i++)
    cout << nums[i] << (i < nums.size() - 1 ? ", " : "");
  cout << "]\n";

  cout << "Output: [";
  for (int i = 0; i < result.size(); i++)
    cout << result[i] << (i < result.size() - 1 ? ", " : "");
  cout << "]\n";

  return 0;
}
```

```
c:\data-structures-cpp\labs\lab-05>cd
\"tempCodeRunnerFile
Input:  [1, 2, 1]
Output: [2, -1, 2]
```

Task 6

```
#include <iostream>
#include <stack>
#include <string>

using namespace std;

struct TagInfo {
  string name;
  int position;
};

int validateHTML(string s) {
  stack<TagInfo> st;

  int i = 0;
  while (i < s.length()) {

    if (s[i] == '<') {
      int startPos = i + 1;

      int j = i + 1;
      while (j < s.length() && s[j] != '>') {
        j++;
      }

      string tagContent = s.substr(i + 1, j - i - 1);
```

```
if (tagContent.back() == '/') {

}

else if (tagContent[0] == '/') {
  string tagName = tagContent.substr(1);
  if (st.empty()) {
    return startPos;
  }

  TagInfo topTag = st.top();
  st.pop();

  if (topTag.name != tagName) {
    return startPos;
  }
}

else {

  string tagName = "";
  for (char c : tagContent) {
    if (c == ' ')
      break;
    tagName += c;
  }

  st.push({tagName, startPos});
}
```

```
      i = j;
    }
    i++;
  }

  if (!st.empty()) {
    return st.top().position;
  }

  return -1;
}

int main() {
  string s1 = "<a><b></b></a>";
  string s2 = "<a><b></a></b>";

  cout << "Input: " << s1 << " -> ";
  int res1 = validateHTML(s1);
  if (res1 == -1)
    cout << "Valid\n";
  else
    cout << "Error at position " << res1 << "\n";

  cout << "Input: " << s2 << " -> ";
  int res2 = validateHTML(s2);
  if (res2 == -1)
    cout << "Valid\n";
  else
```

```
    cout << "Error at position " << res2 << "\n";


  return 0;
}
```

```
c:\data-structures-cpp\labs\lab-05>cd "c:\dat
tional-task
Input: <a><b></b></a> -> Valid
Input: <a><b></a></b> -> Error at position 7

c:\data-structures-cpp\labs\lab-05>
```